The Identity Token System

Polaris

Fixus inter mutabilia

Egor Khaklin

Seton Hill University · Professor Sami Alshalwi

Contents

1 Project Overview

The database models a unified national identity infrastructure designed to replace the full set of physical and numerical credentials that Americans currently carry across different domains of life. In the present system, a single person may hold a driver's license (state DMV), a passport (federal Department of State), a Social Security card (Social Security Administration), a birth certificate (county health department), a health insurance card (private insurer), a voter registration card (state election authority), and a Real ID, each issued under different authority, with different cryptographic protections or none at all, and each creating its own identity silo. The system modeled here consolidates all of these into a single active credential per person: one physical identity token in the holder's possession at any given time, signed under a post-quantum cryptographic algorithm, and verified by authorized agencies for specific contexts.

The token is physical, durable, and quantum-resistant by design. Each individual may hold multiple provisioned tokens (a one-to-many relationship), but only one is active at any given time; the rest sit in reserve, held against the possibility of loss, damage, or compromise of the active token. If the active token is lost, a reserve token can be activated through an expedited process rather than requiring full reissuance from scratch. A previously active token transitions to dormant status when superseded and is retained for audit. The cryptographic algorithm entity is populated with real post-quantum standards (ML-DSA, SLH-DSA) alongside legacy classical algorithms (ECDSA, RSA) so that queries can distinguish quantum-resistant from deprecated signing material directly in SQL.

Digital projection extends the physical token into the holder's personal devices without introducing cloud storage. A token may authorize bindings on a small number of personal devices (phone, tablet, watch), each of which stores a cryptographic copy in the device's secure enclave and can present the credential in contexts that accept digital presentation. The physical token remains authoritative; device bindings are derivative and independently revocable. No backup credential is ever stored on a server, and no cloud provider holds any part of the token. If a device is lost, its binding can be revoked without affecting the physical token or other device bindings.

Context-scoped verification is the key architectural feature. A single token supports verification for banking, employment, healthcare, travel, voting, motor vehicle operation, and government benefits, but each verification event occurs in exactly one context. The verification context entity carries the minimum security level and biometric requirement per context, and a permission junction determines which contexts a given token is authorized for. A token authorized only for banking and employment cannot be used at a TSA checkpoint; a token authorized only for motor vehicle operation cannot be presented as a passport. The verification layer is where the system generates volume. Every time a bank, employer, hospital, airport, polling station, or government agency verifies someone's identity, a verification event record is created capturing who verified, when, for what context, and with what outcome. This is the transactional core of the database and the source of the most interesting queries.

Two privacy and security features distinguish this design from a naive centralized identity database. First, **zero-knowledge verification** allows most verification events to record that a valid token was presented without recording which token. The `disclosure_level` column on each verification event controls what is stored: `ZERO_KNOWLEDGE` (no token identity, only a cryptographic commitment), `SELECTIVE` (specific attributes disclosed, e.g., age or citizenship but not address), or `FULL` (complete token identity, used only under warrant or with explicit consent). This prevents the verification log from functioning as a comprehensive surveillance database while preserving audit capability for fraud and abuse. Second, **on-device biometric binding** ties each token to its holder through a local biometric check performed by the token hardware itself. The biometric template never leaves the device and is never stored in the database; `IdentityToken` records only the binding type (`FINGERPRINT`, `FACE`, `IRIS`, or `NONE`), enrollment date, witnessing

agency, and liveness-check method used at enrollment. If the token is lost, the biometric template is destroyed with it.

1.1 Why Post-Quantum from Day One

The decision to model post-quantum signing as a first-class property of the schema, rather than as a migration target for a later revision, follows from the time horizon of identity data. Most cryptographically protected information has a finite secrecy horizon: a session key for a session, a transaction for a settlement window, an industrial design for a product cycle. Identity data does not. A name, date of birth, citizenship, biometric binding, and any genomic anchor carry a horizon equal to the holder's lifetime plus the lifetimes of relatives whose genome partially recovers from theirs. The information is multi-generational, and once exposed cannot be rotated, revoked, or reissued.

Mosca's inequality (Mosca, 2018) formalizes the resulting risk. Let X be the secrecy horizon, Y the migration time, and Z the time until a cryptographically relevant quantum computer enters service: if $X + Y > Z$, migration is already behind schedule. For identity records built on biometric or genomic anchors, X falls in the 90–120-year range; for a national-scale infrastructure Y is on the order of a decade; by every public estimate Z is shrinking faster than projected. The inequality is not a future condition but the present one: adversaries running harvest-now-decrypt-later programs are not waiting to attack identity data, they are waiting for the rendering event at which the protection dissolves on its own. A credential system deployed on classical cryptography in 2026 is therefore not a working system requiring future migration; it is a deferred compromise, and the schema is designed accordingly. `CryptographicAlgorithm` is modeled as a queryable entity with `quantum_resistant` and `deprecation_date` columns; `AgencyAlgorithmAuth` bounds which agencies may issue under which algorithms; `predecessor_token_id` chains cryptographic generations forward across reissuance; classical algorithms appear in the schema only to make their deprecation queryable, while new tokens are issued exclusively under post-quantum primitives (ML-DSA, SLH-DSA) per the FIPS 203/204/205 finalization of August 2024.

1.2 Scope at a Glance

- **Ten entities on the ER diagram** (Individual, Agency, CryptographicAlgorithm, VerificationContext, IdentityToken, TokenLifecycleEvent, VerificationEvent, DeviceBinding, BlockchainAnchor, RevocationList) plus **two junction tables** (AgencyAlgorithmAuth, TokenPermission) to resolve the M:N relationships.
- **One active credential per person** (HOLDS is 1:N with a uniqueness constraint). Each person may have multiple tokens provisioned, but only one may be `ACTIVE` at a time. Additional tokens sit in `RESERVE` state for emergency activation; superseded tokens transition to `DORMANT` and are retained for audit.
- **Two one-to-many relationships into IdentityToken** (from Agency as issuer, from CryptographicAlgorithm as signer) and two outward from it (to TokenLifecycleEvent via GENERATES, to VerificationEvent via VERIFIED).
- **Two many-to-many relationships** (Agency↔CryptographicAlgorithm via AUTHORIZED, IdentityToken↔VerificationContext via PERMITTED) each resolved through a dedicated junction table with its own attributes.
- **One high-volume associative entity** (VerificationEvent) recording every verification transaction: which token was verified, by which agency, for what context, with what outcome.
- **Domain constraints** enforced through `CHECK` clauses on status fields to prevent invalid values, and through a partial unique index to enforce one-active-token-per-person.

Scope note: DeviceBinding, BlockchainAnchor, and RevocationList are included as first-class entities because each represents real verifier-facing data that a production system would need to query: on-device digital projections (DeviceBinding), decentralized-identifier anchoring for the alternative trust architecture noted in Limitations (BlockchainAnchor), and verifier-facing historical revocation records for audit-accurate freshness checks (RevocationList). Together they remain within conceptual ER scope as entities directly subsidiary to IdentityToken rather than as implementation details deferred to the relational model.

1.3 Reading Guide

The body of this report follows the standard database-design pipeline. Section 2 collects the operational requirements through user roles and seven concrete use cases. Section 3 restates those requirements as numbered functional and non-functional specifications. Sections 4 through 8 then trace the design forward through the standard sequence: information analysis identifies the entities and relationships, ER modeling formalizes them in Chen notation, relational mapping derives the schema (with a normalization proof in 6.5 establishing that every relation is in BCNF), SQL implementation realizes it, and relational algebra expresses representative queries. Each stage is the input to the next, and the use cases introduced at the start are referenced at every subsequent stage so that the operational pattern and the schema-level construct that supports it can be read against each other.

The appendices are not afterthoughts. They are the dimensions in which the body's claims are tested. Appendix A formalizes the lifecycle invariants that the schema's `CHECK` constraints alone cannot express. Appendix B addresses the query-plan and partitioning concerns that determine whether the schema can carry national-scale verification volume. Appendix C situates the design against a federated national eID, Aadhaar, mDL, EU eIDAS 2.0, and the W3C verifiable-credentials stack, locating the distinguishing feature (`disclosure_level` as a schema-level mechanism) against five existing approaches. Appendix D runs the schema through STRIDE and names the three threats the database layer cannot mitigate. Appendix E formalizes the deeper architectural argument: cryptographic primitives are the substrate of digital sovereignty, and every higher-level property is derivative of them. Appendix F sketches the two horizons beyond fingerprint-, face-, and iris-binding (genomic and quantum-observer) and the schema extensions each would require.

A reader evaluating the database deliverable can read sections 2 through 8 and Appendices A and B; a reader evaluating the architectural argument should add Appendices C through F.

2 Information Collection

2.1 Problem Domain

The chosen problem domain is **national identity credentialing**. The current US system distributes identity authority across federal, state, county, and private issuers, with each credential carrying a different cryptographic posture (or none), serving a different verification context, and creating its own identity silo. The system specified here consolidates all civic identity into a single quantum-resistant hardware token per individual while preserving the federation of issuing authorities and the per-context verification model already in operation.

Information was gathered through analysis of: (1) current US credential issuance practices across federal, state, and county agencies; (2) NIST post-quantum cryptography standards (FIPS 203 for key encapsulation, FIPS 204 for digital signatures based on ML-DSA, FIPS 205 for stateless hash-based signatures based on SLH-DSA), finalized August 2024; (3) zero-knowledge proof literature, including the disclosure-level taxonomy used in selective-disclosure verifiable credentials; (4) the W3C Decentralized Identifier specification for the optional ledger-anchoring path;

and (5) failure-mode analysis of relevant historical incidents (the 2017 Equifax breach as a case study in centralized identity-data concentration; the 2020 SolarWinds compromise as a case study in supply-chain risk to issuing authorities; historical denaturalization programs as a case study in issuer-discretion abuse).

2.2 User Roles

Five user roles interact with the database. Each has a distinct access pattern, summarized below.

Table 1: User roles and their database access patterns.

Role	Description and Database Access Pattern
Individual (Token Holder)	The natural person enrolled in the system. Does not directly write to the database. Indirectly triggers `VerificationEvent` inserts whenever they present a token at a verifier and triggers `DeviceBinding` inserts when they bind a personal device. May initiate reserve activation through an issuing agency.
Issuing Agency Personnel	Authorized staff at federal, state, county, or private issuing authorities. Inserts into `Individual` (during initial enrollment) and `IdentityToken`; writes to `TokenLifecycleEvent` on every state change; writes to `RevocationList` when revoking. Subject to the agency's authorization level and `AgencyAlgorithmAuth` entries.
Verifying Agency Personnel	Authorized staff or automated systems at any verifier (banks, hospitals, airports, polling stations, employers, government benefit offices). Reads from `IdentityToken` and `TokenPermission` to validate; writes a single `VerificationEvent` record per verification at the appropriate `disclosure_level`. Cannot modify token state.
System Administrator	Manages the schema itself, the `CryptographicAlgorithm` table, and `AgencyAlgorithmAuth` authorization grants. Identifies tokens at risk from algorithm deprecation and orchestrates migration cohorts. Has read access across all tables for operational monitoring.
Audit / Court Official	Subject to a warrant or explicit holder consent. Reads from `VerificationEvent`, `TokenLifecycleEvent`, and `RevocationList` to reconstruct a token's history for fraud investigation, abuse review, or constitutional challenge. Can only view `FULL` disclosure-level events without further authorization; `ZERO_KNOWLEDGE` events return no identifying information by design.

2.3 Use Cases

The seven use cases below describe how each role interacts with the database during a typical operational year. They cover the full token lifecycle (issuance, activation, verification, device binding, replacement, revocation) and the two cross-cutting administrative concerns (algorithm migration, warrant-authorized audit). Each use case names the database tables touched, the operations performed, and the resulting state changes.

Each entry follows the same structure: **Primary Actor** (the role that initiates the operation), **Secondary Actor** (the role or component that participates passively), **Goal** (the post-condition the operation aims to establish), **Trigger** (the event that initiates the use case), **Main Flow** (the ordered sequence of database operations), **Tables Touched** (the schema surface the operation exercises), and a **Note** field where a constraint or implementation detail deserves explicit mention. The sequence below moves from initial enrollment (UC-1) through the two verification contexts that exercise the disclosure model (UC-2 zero-knowledge, UC-3 full disclosure), into exception handling (UC-4 reserve activation, UC-5 device binding), and concludes with the two administrative use cases (UC-6 algorithm migration, UC-7 warrant-authorized audit) that justify several of the schema's less obvious design decisions.

UC-1: New Token Issuance and Initial Activation

Primary Actor	Issuing Agency Personnel
Secondary Actor	Individual (citizen presenting for enrollment)
Goal	Create a new identity token, bind it to an individual under a quantum-resistant algorithm, and place it in `ACTIVE` state.
Trigger	A citizen presents identity-establishing documents at an authorized issuing agency for the first time, or for re-enrollment following revocation.
Main Flow	1. Issuing officer validates that their agency holds an `ISSUE` or `BOTH` grant in `AgencyAlgorithmAuth` for the chosen algorithm. 2. If the individual has no existing record, `INSERT` a new row into `Individual`. 3. `INSERT` a new row into `IdentityToken` with `status='RESERVE'`, populated FK references to `Individual`, `Agency`, and `CryptographicAlgorithm`. 4. `INSERT` a `TokenLifecycleEvent` of type `ISSUED`. 5. Hardware binding ceremony: officer witnesses biometric enrollment on the token's secure enclave; `UPDATE` the token's biometric metadata fields and `enrollment_witness_agency_id`. 6. `UPDATE IdentityToken` setting `status='ACTIVE'` and `activated_date`; the partial unique index enforces no other active token exists for this individual. 7. `INSERT` a `TokenLifecycleEvent` of type `ACTIVATED`. 8. `INSERT` default `TokenPermission` rows for the contexts the holder is eligible for.
Tables Touched	`Individual`, `IdentityToken`, `TokenLifecycleEvent`, `TokenPermission`, `AgencyAlgorithmAuth` (read-only check)

UC-2: Banking Verification (Zero-Knowledge)

Primary Actor	Verifying Agency Personnel (bank, automated kiosk, or online banking system)
Secondary Actor	Individual presenting their token
Goal	Confirm the holder presents a valid token authorized for `BANKING` without recording the token's identity in the verification log.
Trigger	A customer initiates a transaction or account-access action that requires identity attestation.

Main Flow

1. Verifier system reads `IdentityToken` to confirm `status='ACTIVE'` and the signing `CryptographicAlgorithm` is not deprecated.
2. Verifier system checks `TokenPermission` for an entry where `context_id` corresponds to `BANKING` and `permission_level` is sufficient.
3. Token hardware produces a zero-knowledge proof of validity that does not reveal `token_id`; the proof commitment is stored in `proof_commitment`.
4. `INSERT` a `VerificationEvent` with `token_id=NULL`, `disclosure_level='ZERO_KNOWLEDGE'`, and the cryptographic commitment retained for fraud-investigation correlation.

Tables Touched `IdentityToken` (read), `TokenPermission` (read), `VerificationEvent` (insert), `VerificationContext` (read)

UC-3: Travel Verification at TSA / Border Crossing (Full Disclosure)

Primary Actor	Verifying Agency Personnel (TSA officer, CBP agent)
Secondary Actor	Individual presenting their token at a security checkpoint
Goal	Confirm the holder's full identity for a context that legally requires identification under federal authority.
Trigger	The holder presents at a designated travel checkpoint where federal law authorizes full identity verification.

Main Flow

1. Verifier system reads `IdentityToken` to confirm `status='ACTIVE'` and biometric binding is appropriate to context (`TRAVEL` requires biometric per `VerificationContext`).
2. Token hardware performs a liveness-checked biometric match locally; result is binary, no template leaves the device.
3. Verifier system checks `TokenPermission` for `TRAVEL` authorization.
4. `INSERT` a `VerificationEvent` with full `token_id` reference, `disclosure_level='FULL'`, and the verifier's location captured in `requestor_location`.

Tables Touched `IdentityToken` (read), `TokenPermission` (read), `VerificationContext` (read), `VerificationEvent` (insert)

UC-4: Reserve Token Activation After Loss

Primary Actor	Issuing Agency Personnel
Secondary Actor	Individual reporting loss of their active token
Goal	Revoke the lost token and promote a pre-provisioned reserve to `ACTIVE` without requiring full reissuance from scratch.
Trigger	The holder reports loss, theft, damage, or compromise of their active token to an authorized issuing agency.

Main Flow

1. Officer authenticates the individual through alternative means (in-person identification, secondary biometric, sworn statement).
2. `UPDATE` the lost token's `IdentityToken` record setting `status='LOST'` or `'REVOKED'`.
3. `INSERT` a `RevocationList` record with appropriate `reason_code` (`LOST`, `STOLEN`, `COMPROMISED`) and a `published_location` URL.
4. `INSERT` a `TokenLifecycleEvent` of type `LOST` or `REVOKED`.
5. Select an existing `IdentityToken` for the same individual where `status='RESERVE'`.
6. `UPDATE` the reserve token: set `status='ACTIVE'`, populate `predecessor_token_id` with the lost token's `token_id`, increment `activation_sequence`, set `activated_date`.
7. `INSERT` a `TokenLifecycleEvent` of type `ACTIVATED` on the new active token.

Tables Touched `IdentityToken` (update), `RevocationList` (insert), `TokenLifecycleEvent` (insert)

Note The partial unique index on `(individual_id) WHERE status='ACTIVE'` permits both the `RESERVE` insert (step 3) and the `ACTIVE` update (step 6) within a single transaction: at step 3 the new row has status `RESERVE` (outside the partial-index predicate, no uniqueness check); at step 6 the row transitions into the predicate and the uniqueness check fires only then, when the holder has no other active token.

UC-5: Device Binding (Digital Projection to a Personal Device)

Primary Actor Individual

Secondary Actor The personal device's secure enclave (cryptographic counterparty)

Goal Authorize a personal phone, tablet, or watch to present the token in contexts that accept digital presentation, without storing any credential material in the cloud.

Trigger The holder initiates device-binding from their physical token through a tap-to-pair or NFC handshake.

Main Flow

1. Token hardware generates a device-specific cryptographic projection bound to the device's secure-enclave fingerprint.
2. `INSERT` into `DeviceBinding` with `token_id`, `device_type`, the secure-enclave-attested `device_fingerprint`, the `binding_method` (`SECURE_ENCLAVE`, `TITAN_SECURITY`, etc.), and an `expires_date`.
3. `INSERT` a `TokenLifecycleEvent` of type `DEVICE_BOUND`.

Reverse Flow (revocation) On device loss, `UPDATE` the binding to `status='REVOKED'` with a `revocation_reason` and `INSERT` a `DEVICE_REVOKED` lifecycle event. The physical token and other device bindings are unaffected.

Tables Touched `DeviceBinding` (insert/update), `TokenLifecycleEvent` (insert)

UC-6: Algorithm Migration Audit

Primary Actor	System Administrator
Goal	Identify the cohort of tokens whose signing algorithm is approaching deprecation so that a coordinated reissuance effort can be scheduled before the deprecation date passes.
Trigger	A scheduled quarterly review, or an emergency NIST advisory shortening an algorithm's deprecation window.
Main Flow	1. Run a join query against `IdentityToken`, `Individual`, and `CryptographicAlgorithm` filtered by `status='ACTIVE'` and a `deprecation_date` within the operational horizon (typically 24 months). 2. Group results by `issuing_agency_id` to allocate reissuance load to the agencies that must perform it. 3. Output is a worklist of (token, holder, current algorithm, target algorithm) tuples that drives the reissuance schedule.
Tables Touched	`IdentityToken` (read), `Individual` (read), `CryptographicAlgorithm` (read), `Agency` (read)
Note	Reissuance itself is implemented as UC-1 followed by UC-4 (lost-token flow) for each affected token, with `predecessor_token_id` chaining the cryptographic generations.

UC-7: Warrant-Authorized Verification History Review

Primary Actor	Audit / Court Official
Secondary Actor	The token holder under investigation
Goal	Reconstruct a token's verification history under valid legal authority, returning only the records the disclosure model permits.
Trigger	A court-issued warrant naming a specific `individual_id`, presented through the audit interface.
Main Flow	1. Auditor presents warrant credentials; the audit interface validates the warrant scope (specific individual, specific time window, specific context if narrowed). 2. Run a join query against `VerificationEvent`, `IdentityToken`, `Individual`, and `VerificationContext` filtered by the warrant's parameters. 3. Apply the disclosure filter: events with `disclosure_level='ZERO_KNOWLEDGE'` return no token-identifying data and are tallied only as aggregate counts; `SELECTIVE` events return the disclosed attributes; `FULL` events return complete records. 4. Augment the result with the corresponding `TokenLifecycleEvent` and `RevocationList` entries to produce a complete history.

Tables Touched	`VerificationEvent`, `TokenLifecycleEvent`, `RevocationList`, `IdentityToken`, `Individual`, `VerificationContext` (all read)
Note	The disclosure model is the architectural protection against the verification log functioning as a comprehensive surveillance database. The schema permits the warrant query; the disclosure level governs what the warrant query can return.

These seven use cases collectively exercise ten of the schema's twelve tables and the principal relationships among them. `AgencyAlgorithmAuth` is read by UC-1 as an authorization gate but is not written by any use case (entries are populated administratively rather than as a side effect of issuance or verification); `BlockchainAnchor` is exercised at the SQL layer rather than through the operational use cases above, since the optional DID-anchoring path is implementer-driven rather than user-driven. The sections that follow specify the database requirements, analyze the entities and relationships in detail, formalize them in a Chen-notation ER diagram, map them to a relational schema, implement them in SQL, and express representative queries from these use cases in relational algebra.

2.4 Use-Case Coverage Matrix

The matrix below shows which tables each use case reads from (R) and writes to (W). The pattern verifies that every table appears in at least one use case and that every use case touches at least two tables. The schema is neither under-utilized nor over-fragmented.

Table 9: Use cases mapped to tables (R = read, W = write).

	Indv.	Agcy.	Alg.	Tok.	TLE	VE	Ctx.	Perm.	Other
UC-1	W	R	R	W	W	—	—	W	—
UC-2	—	R	—	R	—	W	R	R	—
UC-3	—	R	—	R	—	W	R	R	—
UC-4	—	—	—	W	W	—	—	—	RevList (W)
UC-5	—	—	—	R	W	—	—	—	DevBind (W)
UC-6	R	R	R	R	—	—	—	—	—
UC-7	R	—	—	R	R	R	R	—	RevList (R)

`IdentityToken` appears in all seven use cases, confirming its role as the architectural hub. `Agency` (read in four cases as either issuer, verifier, witness, or actor) and `TokenLifecycleEvent` (written in four cases as the audit-trail backbone) follow at the next tier. `VerificationEvent` dominates the verification-context use cases (UC-2 and UC-3) and is the high-volume transactional table. The pattern of writes reveals two entry points to the schema: enrollment (UC-1) and verification (UC-2, UC-3); everything else is downstream lifecycle administration. The read/write asymmetry sorts the use cases into three load profiles the schema must satisfy under production traffic: read-heavy, low-latency verification (UC-2, UC-3, producing one append-only row each); write-heavy issuance (UC-1, writing to four tables in a single transaction); and read-only audit (UC-7, reconstructing narratives across six tables without modifying any).

3 Database Requirements Document

3.1 Project Scope

The *Identity Token System* is a unified national identity infrastructure designed to replace the current fragmented set of physical and digital credentials (driver's licenses, passports, Social Security cards, birth certificates, Real ID, voter registration cards, health insurance cards) with a single quantum-resistant hardware token per citizen. The system supports context-scoped verification across agencies and jurisdictions while preserving strong privacy guarantees through zero-knowledge and selective disclosure mechanisms. The relational database specified in this report is the persistent backbone of that infrastructure: it stores the issuance, lifecycle, authorization, and verification records that make federated trust auditable while keeping biometric templates and unnecessary verification metadata out of the database entirely.

3.2 Functional Requirements

- One active credential per person, with reserve tokens for redundancy and expedited recovery (FR-1).
- Context-scoped verification (`BANKING`, `EMPLOYMENT`, `HEALTHCARE`, `TRAVEL`, `VOTING`, `MOTOR_VEHICLE`, `GOVERNMENT_BENEFITS`) with per-context biometric and security-level requirements (FR-2).
- Zero-knowledge verification (`ZERO_KNOWLEDGE`, `SELECTIVE`, `FULL` disclosure levels) to prevent the verification log from becoming a surveillance database (FR-3).
- On-device biometric binding (`FINGERPRINT`, `FACE`, `IRIS`, `NONE`) with no biometric templates stored centrally (FR-4).
- Full append-only audit trail via `TokenLifecycleEvent` and `VerificationEvent` (FR-5).
- Optional post-quantum decentralized-identifier (DID) anchoring on distributed ledgers (FR-6).
- Agency authorization matrix controlling which agencies may issue or verify under each cryptographic algorithm (FR-7).
- Token succession tracking through a self-referential `predecessor_token_id` foreign key, capturing the cryptographic lineage when a reserve is activated or a token is reissued (FR-8).
- Verifier-facing revocation publication, distinct from the token's own status field, to support audit-accurate freshness checks (FR-9).

3.3 Non-Functional Requirements

- Quantum resistance using ML-DSA and SLH-DSA (NIST FIPS 204 and 205 post-quantum standards) as primary signature algorithms; legacy classical algorithms (ECDSA, RSA) retained only for queryable deprecation tracking (NFR-1).
- High-volume transactional support: the verification layer must accept thousands of `VerificationEvent` inserts per day per major verifier without locking contention (NFR-2).
- Strict referential integrity enforced through foreign keys on every cross-entity reference; `CHECK` constraints on every enumerated field; partial uniqueness on the active-token invariant (NFR-3).
- Auditability: every token state transition produces a `TokenLifecycleEvent`; the lifecycle table is append-only by convention and tooling, not by storage engine (NFR-4).
- Support for token lifecycle states (`ACTIVE`, `RESERVE`, `DORMANT`, `REVOKED`, `LOST`, `EXPIRED`) with constrained transitions enforced at the application layer (NFR-5).

- Privacy-preserving by construction: no biometric template, no verification location more granular than necessary, and no token identity stored on `ZERO_KNOWLEDGE` verification events (NFR-6).

4 Information Analysis

This section identifies the entities, relationships, and key constraints that the requirements imply, before formalizing them as an ER diagram in the next section. The analysis distinguishes *conceptual* entities (independent things the database represents) from *associative* entities (records that exist only because two other entities relate to each other) and notes the constraints that must be carried forward into the schema.

4.1 Entity Identification

Ten entities emerge from the requirements. They divide cleanly into three groups: **principals** (the actors and authorities), **the central artifact** (the token itself), and **records** (the events and subsidiary structures that exist relative to a token).

Table 10: Entities identified during requirements analysis.

Entity	Group	Why It Exists
Individual	Principal	The natural person whom the token credentials.
Agency	Principal	The federal, state, county, or private authority that issues or verifies.
CryptographicAlgorithm	Principal	The signing primitive used by tokens; modeled as a first-class entity so deprecation and quantum-resistance status are queryable.
VerificationContext	Principal	The seven fixed contexts in which verification can occur; modeled as an entity rather than an enum so per-context security and biometric requirements can be queried and joined.
IdentityToken	Central artifact	The credential itself; every other relationship in the schema either originates from or terminates at this entity.
TokenLifecycleEvent	Record	Append-only log of state transitions on a single token.
VerificationEvent	Record	The high-volume transactional record produced every time a token is presented for verification.
DeviceBinding	Record	A digital projection of a physical token to a personal device's secure enclave.
BlockchainAnchor	Record	The optional ledger-anchored DID commitment for the alternative trust architecture.
RevocationList	Record	Verifier-facing historical record of revocations, separating revocation publication from token status.

4.2 Relationship Identification

Thirteen relationships connect these ten entities, including one self-referential link on `IdentityToken`. The pattern is hub-and-spoke: `IdentityToken` is the hub, and most other entities relate to it directly. The single peer-to-peer relationship is `Agency` AUTHORIZED `CryptographicAlgorithm`, the authorization matrix that governs which agencies may use which signing algorithms.

- **1:N relationships into `IdentityToken`** from `Individual` (HOLDS), `Agency` (ISSUES), and `CryptographicAlgorithm` (SIGNS).

- **1:N relationships out of `IdentityToken`** to `TokenLifecycleEvent` (GENERATES), `VerificationEvent` (VERIFIED), `DeviceBinding` (PROJECTS_TO), and `RevocationList` (REVOKED_VIA).
- **0..1:1 relationship** from `IdentityToken` to `BlockchainAnchor` (ANCHORED_TO), optional on the token side: a token may have zero or one anchor; an anchor always references exactly one token.
- **M:N relationship** between `Agency` and `CryptographicAlgorithm` (AUTHORIZED), resolved in the relational model to junction table `AgencyAlgorithmAuth`.
- **M:N relationship** between `IdentityToken` and `VerificationContext` (PERMITTED), resolved in the relational model to junction table `TokenPermission`.
- **Self-referential 0:1** on `IdentityToken` via `predecessor_token_id`, capturing token succession.
- **Two FK references into `VerificationEvent`** from `Agency` (`requesting_agency_id`) and `VerificationContext` (`context_id`), each 1:N.

4.3 Key Constraints Identified

Four constraints emerged as architecturally important during analysis. Each is preserved through the ER diagram (where it is annotated) and the relational model (where it becomes an explicit schema constraint).

- **One active token per individual.** The relationship `Individual` HOLDS `IdentityToken` is 1:N at the level of provisioned tokens, but a partial uniqueness rule restricts it to 1:1 over the subset where `status='ACTIVE'`. This is implemented in SQL as a partial unique index on `individual_id` where `status='ACTIVE'`. UC-4 (reserve activation) avoids any need for deferred constraint checking by ordering the swap: the lost token transitions to its terminal status *first* (releasing it from the partial index's predicate), then the reserve is promoted to `ACTIVE`.
- **Dual role of `Agency`.** An `Agency` record may act as an issuer, a verifier, a witness, a revoker, or a lifecycle-event actor. The role is determined by which foreign-key column holds the reference: `issuing_agency_id` and `enrollment_witness_agency_id` on `IdentityToken`, `requesting_agency_id` on `VerificationEvent`, `actor_agency_id` on `TokenLifecycleEvent`, and `revoked_by_agency_id` on `RevocationList`. The agency record itself is identical across all roles. This is the standard ER pattern for multiple roles on a single entity, preserved through to the SQL schema.
- **Nullable `token_id` on `VerificationEvent`.** Verification events at `disclosure_level='ZERO_KNOWLEDGE'` explicitly do not record which token was verified. The FK on `VerificationEvent.token_id` is therefore nullable, by design. This is the schema-level mechanism that prevents the verification log from functioning as a surveillance database.
- **Self-referential FK on `IdentityToken`.** The `predecessor_token_id` column references `IdentityToken` itself, capturing which token was superseded when a reserve was activated or a reissuance occurred. The reference is nullable (the first token in any holder's sequence has no predecessor) and forms a chain that can be walked recursively to reconstruct the full lineage of a holder's tokens.

4.4 Participation Constraints

Participation constraints record whether each entity is required (`TOTAL`) or optional (`PARTIAL`) in each relationship. The summary below records the constraint on both sides of every relationship and notes the schema-level mechanism that enforces it.

- **Total on both sides:** GENERATES (every token has at least one lifecycle event, beginning with the `ISSUED` entry created during UC-1; every event references exactly one token through a `NOT NULL` foreign key).
- **Partial on left, total on right:** HOLDS (an `Individual` record may exist without a current token in three legitimate scenarios: David's revoked sample case, the transient state during UC-1 between the `Individual` insert and the `IdentityToken` insert, and the period after a token's terminal transition before reissuance — but every token has exactly one holder, enforced by `NOT NULL` on `individual_id`); ISSUES, SIGNS, PROJECTS_TO, ANCHORED_TO, REVOKED_VIA (verifier-only agencies do not issue, deprecated algorithms sign no current tokens, and tokens need not bind any device, anchor any DID, or be revoked — but every record on the right side always references exactly one token); AUTHORIZED (a newly-registered or dormant `Agency` may carry no algorithm grants until it begins operations); PERMITTED (`RESERVE` and terminal-state tokens carry no permission grants; permissions are issued at activation per UC-1 step 8).
- **Partial on both sides:** VERIFIED — a token may have zero verifications during a quiet period, and a verification event may carry `token_id=NULL` under `ZERO_KNOWLEDGE` disclosure. This bilateral partial participation is the schema's most distinctive constraint and the architectural mechanism that prevents the verification log from functioning as a surveillance database.

The `TOTAL` participation cases on the `IdentityToken` side (HOLDS, ISSUES, SIGNS, GENERATES) are enforced at the schema level through `NOT NULL` on the corresponding foreign-key columns. `PARTIAL` cases leave those columns nullable. The remaining totality on GENERATES (every token has at least one lifecycle event) is enforced operationally rather than schematically: the application layer pairs token issuance with at least one `TokenLifecycleEvent` insert in the same transaction, and the append-only trigger on the lifecycle table prevents the event from later being deleted.

Three enforcement layers correspond to three classes of constraint. Single-row constraints (one row per (token, holder) pair, one row per (token, algorithm) pair, valid status values) are expressible as `NOT NULL`, `UNIQUE`, and `CHECK` clauses directly on table definitions and are enforced by the storage engine at the time of every `INSERT` or `UPDATE`. Cross-row invariants over a single table (one `ACTIVE` token per individual, the legal state-transition set) are expressible as partial unique indices and triggers; they require the storage engine to evaluate predicates against the current table state, which is more expensive but still local. Cross-table invariants that span multiple statements (every token issuance must be accompanied by an `ISSUED` lifecycle event, every reserve activation must produce both a status update and a corresponding event) cannot be expressed as schema constraints in standard SQL; they live in stored procedures and application code, with the database providing only the audit trail necessary to detect violations after the fact. The schema is designed to push as many invariants as possible down into the first two layers, leaving the third layer to enforce only what genuinely requires multi-statement coordination.

5 ER Modeling

This section formalizes the entities and relationships from the analysis above in a Chen-notation ER diagram, accompanied by a complete attribute reference and a relationship catalog with cardinalities and design notes.

5.1 Entity-Relationship Diagram

Figure 1 presents the core seven-entity diagram with `IdentityToken` at the center. Figure 2 shows the three subsidiary entities (DeviceBinding, BlockchainAnchor, RevocationList) that attach directly to `IdentityToken` but are rendered separately to preserve the symmetric layout of the primary diagram.

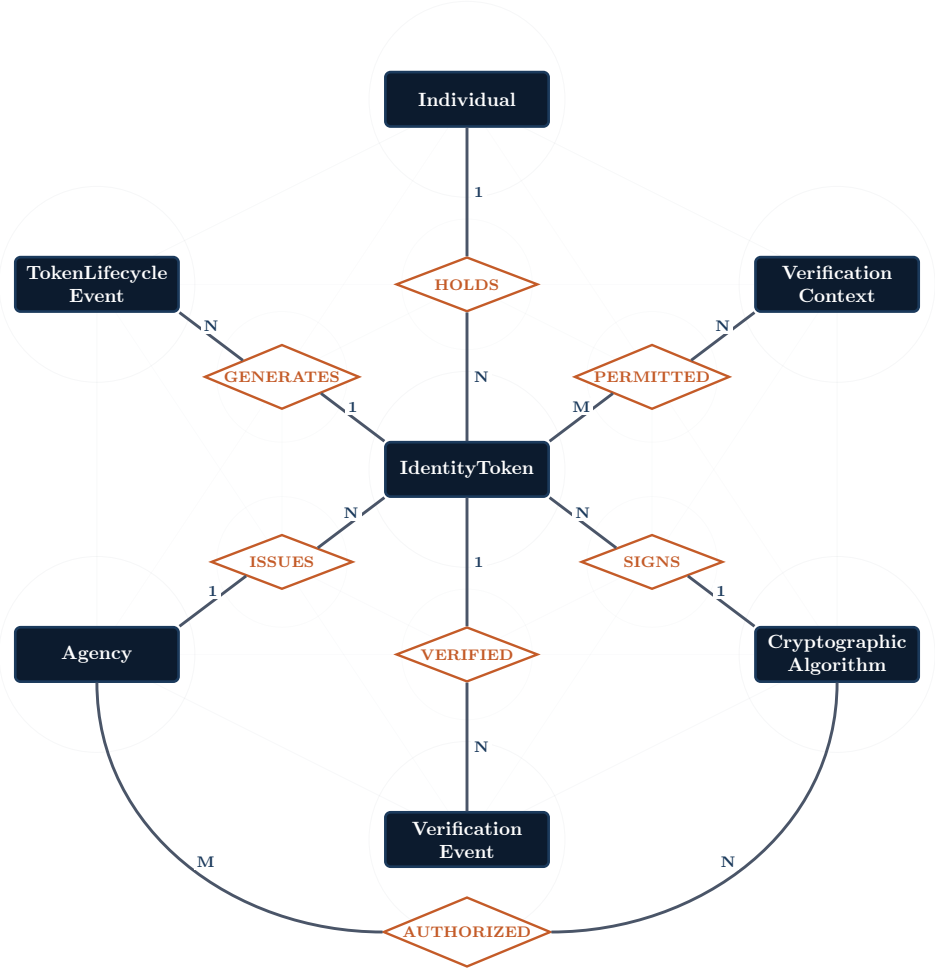

Figure 1: Core entity–relationship diagram in Chen notation. Navy rectangles are entities; orange diamonds are relationships. Cardinality labels (1, N, M) appear on the connecting lines. The seven core entities are arranged symmetrically around `IdentityToken` at the center. The `Individual` to `IdentityToken` relationship is one-to-many with a partial uniqueness constraint (multiple provisioned tokens, one active at a time). The two M:N relationships (AUTHORIZED and PERMITTED) resolve to junction tables in the relational model. Three additional subsidiary entities (BlockchainAnchor, RevocationList, DeviceBinding) attach directly to `IdentityToken` and are shown separately in Figure 2.

Reading the diagram. The diagram encodes three design choices through visual structure. *First*, `IdentityToken` sits at the geometric center because every other entity relates to it through exactly one relationship, making the schema a hub-and-spoke topology rather than a peer mesh. *Second*, the four principals (`Individual`, `Agency`, `CryptographicAlgorithm`, `VerificationContext`) are arranged at the four cardinal positions because they are the schema's

external authorities — entities that exist independently of any token and govern what tokens can do. *Third*, the two record-type entities (`TokenLifecycleEvent`, `VerificationEvent`) are placed at the secondary positions because they are existentially dependent on `IdentityToken` and are the high-volume tables in any real deployment. The single peer-to-peer relationship AUTHORIZED is rendered as the bottom arc connecting `Agency` and `CryptographicAlgorithm`, distinguishing it from the spokes. This layout is not aesthetic: it makes the schema's information-flow direction immediately legible. Reads flow inward (verifiers consult permissions, contexts, algorithms); writes flow outward (issuance creates token, lifecycle event, and permissions in one transaction); cross-cutting administrative queries (UC-6 algorithm migration, UC-7 warrant audit) traverse the whole topology.

Figure 2 below shows three additional subsidiary entities that attach directly to `IdentityToken`. They are rendered separately from Figure 1 because they are structurally peripheral to the core seven-entity model: each exists only to store data derived from or referencing a specific `IdentityToken`. Placing them on the primary diagram would obscure the symmetric hub-and-spoke relationships among the core entities without adding clarity about their own role.

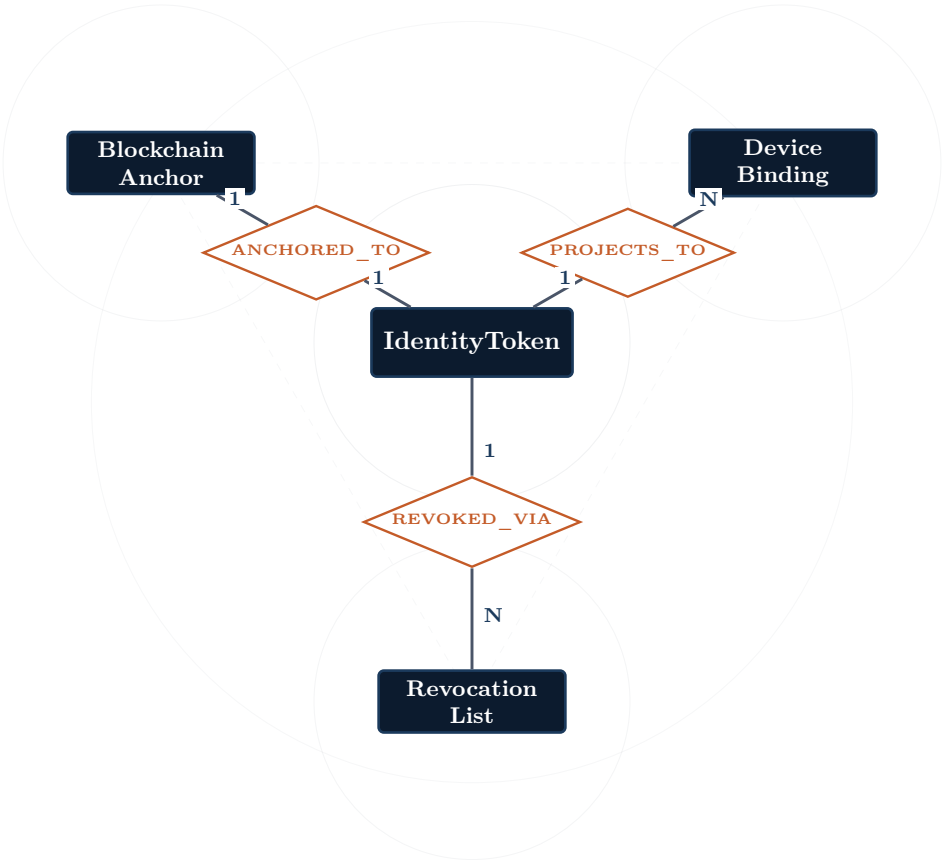

Figure 2: `IdentityToken` and its three subsidiary entities, arranged with triadic rotational symmetry around the central token. `BlockchainAnchor` represents optional decentralized-identifier anchoring on a post-quantum-capable distributed ledger (discussed as an alternative trust architecture in the Limitations section). `RevocationList` is a verifier-facing historical revocation registry, separating revocation publication from the token's own status field so that verifiers can perform audit-accurate freshness checks. `DeviceBinding` represents on-device digital projections of the physical token stored in a device's secure enclave (no cloud storage). All three attach to `IdentityToken` but are shown separately from the core diagram for clarity.

The split between Figures 1 and 2 reflects a design distinction, not a layout convenience. The seven core entities are *constitutive*: each represents a principal that the system is fundamentally about (people, authorities, algorithms, contexts, the credential itself, and the two records the credential generates over its lifetime). Removing any of them collapses the model. The three subsidiary entities are *derivative*: each exists to serve a specific operational requirement that does not arise in the conceptual model itself but does arise once the schema is deployed against real verifiers. `DeviceBinding` exists because verifiers accept digital presentation; `BlockchainAnchor` exists because some trust models prefer ledger-anchored identifiers; `RevocationList` exists because verifier-side freshness checks need a publication channel distinct from the token's own status field. They attach to `IdentityToken` as derivative records rather than as participants in the symmetric hub-and-spoke topology of the core diagram. The two diagrams together specify the complete schema; their separation makes the architectural distinction legible.

5.2 Entity Reference

The ER diagrams above show entity names and relationships only. Complete attribute specifications appear in Table 11. Primary keys are shown in **bold underlined** text; foreign keys are shown in *italic underlined* text.

The attribute set for each entity is the minimum needed to answer the operational queries from §2 and the constraint requirements from §3. Three conventions apply throughout: primary keys are surrogate `SERIAL` integers (natural keys would propagate mutability and sensitivity through the FK structure); enumerated fields appear in italic with their fixed domain listed parenthetically (these are enforced by `CHECK` constraints at the schema level); and sensitive data the system refuses to store — raw biometric templates, plaintext genomic sequences, token-identifying data on `ZERO_KNOWLEDGE` verifications — is conspicuously absent from the attribute lists. The schema's privacy posture is visible in what it refuses to record, not only in what it constrains.

Table 11: Complete attribute reference for all entities.

Entity	Attributes
Individual	individual_id, legal_name, date_of_birth, jurisdiction, enrollment_date
Agency	agency_id, name, agency_type, jurisdiction, authorization_level
CryptographicAlgorithm	algorithm_id, name, family, quantum_resistant, nist_standard, security_level_bits, public_key_size, signature_size, deprecation_date
VerificationContext	context_id, context_type *(BANKING, EMPLOYMENT, HEALTHCARE, TRAVEL, VOTING, MOTOR_VEHICLE, GOVERNMENT_BENEFITS)*, description, requires_biometric, min_security_level
IdentityToken	token_id, token_value, physical_serial, hardware_model, biometric_binding_type *(NONE, FINGERPRINT, FACE, IRIS)*, biometric_enrolled_date, *enrollment_witness_agency_id*, liveness_check_type *(PASSIVE, ACTIVE_CHALLENGE, MULTI_MODAL)*, *individual_id*, *issuing_agency_id*, *algorithm_id*, *predecessor_token_id*, activation_sequence, status *(ACTIVE, RESERVE, DORMANT, REVOKED, LOST, EXPIRED)*, issued_date, activated_date, expiration_date
TokenLifecycleEvent	event_id, *token_id*, *actor_agency_id*, event_type *(ISSUED, ACTIVATED, DEACTIVATED, DEVICE_BOUND, DEVICE_REVOKED, REVOKED, LOST, EXPIRED, REPLACED)*, event_timestamp, reason_code
VerificationEvent	event_id, *token_id* *(nullable)*, *requesting_agency_id*, *context_id*, event_timestamp, outcome *(SUCCESS, FAILURE, EXPIRED, UNAUTHORIZED)*, disclosure_level *(ZERO_KNOWLEDGE, SELECTIVE, FULL)*, proof_commitment, requestor_location
DeviceBinding	binding_id, *token_id*, device_type *(PHONE, TABLET, WATCH)*, device_fingerprint, binding_method, authorized_date, expires_date, status *(ACTIVE, REVOKED)*, revocation_reason
BlockchainAnchor	anchor_id, *token_id*, did, commitment_hash, ledger_network *(ALGORAND_PQ, HYPERLEDGER_INDY, CUSTOM_LATTICE)*, anchor_tx_hash, anchored_date, status *(ACTIVE, SUPERSEDED, REVOKED)*
RevocationList	revocation_id, *token_id*, *revoked_by_agency_id*, revocation_timestamp, effective_date, reason_code *(COMPROMISED, LOST, STOLEN, SUPERSEDED, ADMINISTRATIVE, DEATH)*, published_location
AgencyAlgorithmAuth	agency_id, algorithm_id, authorization_type, authorized_date *(M:N junction for AUTHORIZED)*
TokenPermission	token_id, context_id, permission_level, granted_date *(M:N junction for PERMITTED)*

Note: The two junction tables at the bottom of the table do not appear on the ER diagram because they represent the implementation-level resolution of the M:N relationships AUTHORIZED and PERMITTED. They appear as explicit relations in the relational model and as `CREATE TABLE` statements in the SQL implementation. The `status` column on `IdentityToken` admits the six values shown above (`ACTIVE`, `RESERVE`, `DORMANT`, `REVOKED`, `LOST`, `EXPIRED`); the legal transitions among them are formalized in Appendix A. Tokens are never deleted from the schema. When a token's lifecycle ends, the record is retained in a terminal status for audit purposes, and a new token is issued as a full replacement.

5.3 Relationship Catalog

Table 12: Relationships, cardinalities, and design notes.

Relationship	Cardinality	Notes
Individual HOLDS IdentityToken	1:N	One person, one active credential, plus optional reserves. Enforced by a partial `UNIQUE` index on `individual_id` where `status = 'ACTIVE'`.
Agency ISSUES IdentityToken	1:N	An issuing agency creates many tokens. Each token has exactly one issuer.
CryptographicAlgorithm SIGNS IdentityToken	1:N	One algorithm signs many tokens. Deprecated algorithms define a queryable replacement cohort.
IdentityToken PROJECTS_TO DeviceBinding	1:N	A token may authorize a small number of personal device bindings (typically ≤ 3). Each binding stores a cryptographic copy in the device's secure enclave; nothing is stored in the cloud.
IdentityToken ANCHORED_TO BlockchainAnchor	0..1:1	An optional decentralized-identifier anchor on a post-quantum-capable distributed ledger. The relationship is mandatory on the `BlockchainAnchor` side (every anchor references exactly one token) and optional on the `IdentityToken` side (a token may have zero or one anchor). Holds commitments and references only, never personal or biometric data. Supports the alternative trust architecture noted in Limitations.
IdentityToken REVOKED_VIA RevocationList	1:N	A token may have zero, one, or multiple revocation records. Separating revocation from the token's own status field allows audit-accurate historical freshness checks by verifiers.
IdentityToken → IdentityToken (predecessor)	0:1	Self-referential FK capturing which token this token succeeded when a reserve was activated. Null for the first token in a holder's sequence.
Agency AUTHORIZED CryptographicAlgorithm	M:N	Resolved via `AgencyAlgorithmAuth`. Authorization is type-scoped (ISSUE, VERIFY, BOTH).
IdentityToken PERMITTED VerificationContext	M:N	Resolved via `TokenPermission`. A token may be authorized for multiple contexts at different permission levels.
IdentityToken GENERATES TokenLifecycleEvent	1:N	Append-only audit trail: ISSUED, ACTIVATED, DEACTIVATED, DEVICE_BOUND, DEVICE_REVOKED, REVOKED, LOST, EXPIRED, REPLACED.
IdentityToken VERIFIED VerificationEvent	1:N	High-volume relationship. A token generates thousands of verification events over its operational life.
VerificationContext → VerificationEvent	1:N	Captured as FK (`context_id`). Each verification occurs in exactly one context, which determines the minimum security level required.
Agency → VerificationEvent	1:N	Captured as FK (`requesting_agency_id`). Agency plays both issuer and verifier roles, distinguished by which FK column references it.

5.4 Design Notes

One active credential per person, multiple provisioned. Each individual may have several tokens provisioned and bound to them, but only one may be in `ACTIVE` state at a time, enforced by a partial unique index on `individual_id` where `status='ACTIVE'`. Reserve tokens sit in `RESERVE` state, pre-issued against the possibility of loss or compromise of the active token; activating a reserve transitions it to `ACTIVE` and transitions the former active token to `DORMANT`. The full sequence is preserved through `activation_sequence` and a self-referential `predecessor_token_id` foreign key. **Device projection without cloud storage.** `DeviceBinding` represents phone,

tablet, or watch copies of the physical token, each stored in the device's secure enclave; no backup credential is ever stored on a server, and the database records only binding metadata (device fingerprint, authorization window, revocation status). Losing a phone means revoking its binding without affecting the physical token or other device bindings. **Agency plays dual roles through distinct foreign keys.** An `Agency` row can be referenced as an issuing agency on `IdentityToken` (`issuing_agency_id`) and as a requesting agency on `VerificationEvent` (`requesting_agency_id`); the two roles are disambiguated by which FK column holds the reference, and the `AgencyAlgorithmAuth` junction further constrains which agencies may issue, verify, or both, per algorithm.

Quantum-resistance is a first-class attribute. `CryptographicAlgorithm` stores a `quantum_resistant` boolean alongside algorithm family, NIST standard designation, and security level in bits, enabling direct SQL queries for tokens at risk from harvest-now-decrypt-later attacks. **VerificationContext is a separate entity rather than an enum.** Contexts carry per-context metadata (`requires_biometric`, `min_security_level`) and participate in the M:N `TokenPermission` relationship; modeling context as a string column would forfeit both capabilities. **Junction tables live in the relational model, not the ER diagram.** The two M:N relationships (AUTHORIZED, PERMITTED) are shown as single diamond relationships on the ER diagram following Chen-notation convention; their resolution into `AgencyAlgorithmAuth` and `TokenPermission` occurs when the ER model is mapped to relations.

Disclosure level is a column on the verification record, not a query parameter. The decision to store `disclosure_level` as a persisted attribute of `VerificationEvent` (rather than as runtime configuration on the verifier) is the architectural choice that distinguishes this design from comparable systems (Appendix C). Verifiers cannot retroactively elevate a `ZERO_KNOWLEDGE` record to `FULL` disclosure: the database literally never stored the identifying fields. The `chk_disclosure_token_consistency` check enforces this at insert time, requiring `token_id IS NULL` for `ZERO_KNOWLEDGE` events and `token_id IS NOT NULL` for `FULL` events. **Lifecycle and verification events are append-only by tooling.** NFR-4 requires every state change to be auditable; `TokenLifecycleEvent` and `VerificationEvent` are append-only at the trigger layer (the reference implementation blocks `UPDATE` and `DELETE` on both), with administrative deletion requiring explicit trigger removal — a deliberate friction point that produces an auditable record of any override.

6 Relational Model

6.1 Entity-to-Table Mapping

Each of the ten entities maps directly to a table. The two many-to-many relationships are resolved via junction tables with composite primary keys, producing twelve tables in total.

Table 13: Entity-to-table mapping.

ER Entity	Relational Table	Notes
`Individual`	`Individual`	Core person record
`Agency`	`Agency`	Dual role (issuer + verifier)
`CryptographicAlgorithm`	`CryptographicAlgorithm`	Post-quantum + classical support
`VerificationContext`	`VerificationContext`	7 fixed contexts with metadata
`IdentityToken`	`IdentityToken`	**Central hub** with multiple FKs
`TokenLifecycleEvent`	`TokenLifecycleEvent`	Append-only audit trail
`VerificationEvent`	`VerificationEvent`	High-volume transactional table
`DeviceBinding`	`DeviceBinding`	1:N subsidiary
`BlockchainAnchor`	`BlockchainAnchor`	Optional DID anchoring (1:1)
`RevocationList`	`RevocationList`	Historical revocation records
`AgencyAlgorithmAuth`	`AgencyAlgorithmAuth`	Junction (M:N AUTHORIZED)
`TokenPermission`	`TokenPermission`	Junction (M:N PERMITTED)

6.2 Relationship Mapping

One-to-Many (1:N). Each 1:N relationship is implemented by placing a foreign key on the table corresponding to the *many* side. `IdentityToken` therefore carries `individual_id` (from HOLDS), `issuing_agency_id` (from ISSUES), `algorithm_id` (from SIGNS), and `enrollment_witness_agency_id` (a secondary reference into `Agency`). `TokenLifecycleEvent`, `VerificationEvent`, `DeviceBinding`, `BlockchainAnchor`, and `RevocationList` each carry `token_id`. `VerificationEvent` additionally carries `requesting_agency_id` and `context_id`; `TokenLifecycleEvent` carries `actor_agency_id`; `RevocationList` carries `revoked_by_agency_id`. The `token_id` on `VerificationEvent` is nullable so that `ZERO_KNOWLEDGE` verifications produce no token-identifying record.

Many-to-Many (M:N). Each M:N relationship resolves to a junction table with a composite primary key on the two FK columns, plus relationship-specific attributes. `AgencyAlgorithmAuth` has primary key `(agency_id, algorithm_id)` and carries `authorization_type` (`ISSUE`, `VERIFY`, or `BOTH`). `TokenPermission` has primary key `(token_id, context_id)` and carries `permission_level` (`READ`, `VERIFY`, or `FULL`).

Self-referential (0:1). `IdentityToken.predecessor_token_id` references `IdentityToken.token_id` on the same table. The reference is nullable; the first token in any holder's sequence has no predecessor.

One-to-one (1:1). `BlockchainAnchor` carries `token_id` as a foreign key with the application-level invariant that at most one anchor exists per token in `ACTIVE` status (additional anchors are permitted in `SUPERSEDED` or `REVOKED` status).

6.3 Schema-Level Constraints Summary

Four schema-level constraints carry architectural weight. **Partial uniqueness** on `individual_id` where `status='ACTIVE'` is implemented as a partial `UNIQUE` index, restricting each individual to one active token at a time while permitting any number of `RESERVE`, `DORMANT`, and terminal-state tokens. **`CHECK` constraints** on every enumerated field guarantee that no invalid status, type, or context value can be inserted (`status` on `IdentityToken`, `outcome` and `disclosure_level` on `VerificationEvent`, `event_type` on `TokenLifecycleEvent`, etc.). **Foreign-key integrity** is enforced on every cross-entity reference; cascades are explicitly avoided so that audit records persist even if their parent token is removed (in practice, tokens are never removed, only transitioned through `REVOKED`, `LOST`, or `EXPIRED`). **Indexes** on high-query columns (`individual_id`, `status`, `event_timestamp`, `context_id`, `token_id` on lifecycle and verification tables) support the verification and audit query patterns.

6.4 Mapping Rules Applied

The relational model above derives from five standard ER-to-relational mapping rules (Elmasri & Navathe, ch. 9), each applied without modification. **Rule 1 (strong-entity mapping):** each of the ten ER entities maps to a table with the entity's primary key as table primary key. **Rule 2 (weak-entity mapping):** no weak entities exist in this schema; all record-type entities (`TokenLifecycleEvent`, `VerificationEvent`, `DeviceBinding`, `BlockchainAnchor`, `RevocationList`) carry their own surrogate primary key. **Rule 3 (1:1 relationships):** the single 0..1:1 (`IdentityToken` ANCHORED_TO `BlockchainAnchor`) places the FK on the optional side. **Rule 4 (1:N relationships):** FK on the *many* side, applied seven times across `IdentityToken`, `TokenLifecycleEvent`, `VerificationEvent`, `DeviceBinding`, and `RevocationList`. **Rule 5 (M:N relationships):** junction table with composite primary key, applied twice (`AgencyAlgorithmAuth` resolving AUTHORIZED; `TokenPermission` resolving PERMITTED), each carrying relationship-specific attributes (`authorization_type` and `permission_level`). The self-referential FK on `IdentityToken` (`predecessor_token_id`) is a direct application of Rule 4 with the same table on both sides. The result is twelve tables, eighteen foreign keys (including the self-referential `predecessor_token_id`), fourteen `CHECK` constraints, and one partial unique index. The full inventory is enumerated in the SQL implementation that follows.

6.5 Normalization Analysis

The mapping rules produce a schema; they do not prove the schema is well-formed. This subsection establishes that every relation is in Boyce-Codd Normal Form (BCNF). The proof proceeds by enumerating the functional dependencies (FDs) on each relation and confirming the left-hand side of every nontrivial FD is a superkey.

First Normal Form. Every attribute holds an atomic value drawn from a defined domain. Multi-valued or composite attributes are not used: the seven `VerificationContext` values are rows in a separate entity rather than a comma-separated string; multiple device bindings per token are rows in `DeviceBinding`; M:N relationships are resolved through junction tables. 1NF holds by construction.

Functional dependencies. The nontrivial FDs fall into two patterns. On every entity table, the surrogate primary key determines all other attributes (`token_id` → all other columns of `IdentityToken`; `individual_id` → all other columns of `Individual`; and so on). No other nontrivial FDs hold because the primary key is surrogate (a system-generated `SERIAL`) rather than derived from any application attribute, so no application attribute determines any other on the same row. On the two junction tables, the composite key determines the relationship-specific attributes: `(agency_id, algorithm_id)` → `(authorization_type, authorized_date)` on `AgencyAlgorithmAuth`, and `(token_id, context_id)` → `(permission_level, granted_date)` on `TokenPermission`.

2NF, 3NF, and BCNF. 2NF requires that no non-key attribute depend on a proper subset of a composite key. On both junction tables the non-key attributes are properties of the relationship rather than of either key column alone, so no partial dependency exists. 3NF additionally requires no transitive dependency: every non-key attribute is determined directly by the primary key, so 3NF holds. BCNF strengthens 3NF by requiring that the left-hand side of every nontrivial FD be a superkey; since the only nontrivial FDs are those listed above, and each left-hand side is the primary key (a superkey by definition), BCNF holds on every relation. The schema is in BCNF without further decomposition. The `UNIQUE (individual_id, status) WHERE status='ACTIVE'` constraint does not affect this analysis: it is a *conditional* uniqueness implementing an application invariant as a partial index, not a relational dependency in the formal sense.

Apparent denormalization, addressed. Two columns warrant explicit comment. `TokenLifecycleEvent.actor_agency_id` appears redundant against `IdentityToken.issuing_agency_id` but is not: the actor on a lifecycle event is the agency that performed the specific transition, frequently *not* the issuing agency (a revocation may be performed by a different jurisdiction; a device-binding event has no agency actor and the column is nullable). The two columns capture different facts, no FD holds between them, and the relation remains in BCNF. `VerificationEvent.requesting_agency_id` and `context_id` are similarly independent: one agency verifies across many contexts and one context is exercised by many agencies.

6.6 Sample DML Operations

The mapping rules above are necessary but not sufficient: the schema must also be exercisable through transactional DML that preserves every constraint. The two listings below show how the use cases from the requirements analysis translate into SQL, demonstrating that the schema operates correctly under realistic write patterns.

UC-1 (new token issuance) as a single transaction. The active-token uniqueness invariant requires the `INSERT` into `IdentityToken` and the paired `TokenLifecycleEvent` to commit together. The partial unique index on `individual_id` where `status='ACTIVE'` permits this naturally: the new token enters at status `RESERVE` (outside the index predicate, no uniqueness check) before the activation update transitions it into `ACTIVE`.

```
BEGIN;
  INSERT INTO Individual (legal_name, date_of_birth, jurisdiction, enrollment_date)
  VALUES ('A. Holder', '2000-04-15', 'US-PA', CURRENT_TIMESTAMP)
  RETURNING individual_id;
  -- assume returned individual_id = 6

  INSERT INTO IdentityToken (token_value, physical_serial, hardware_model,
                             biometric_binding_type, individual_id,
                             issuing_agency_id, algorithm_id, status,
                             activated_date, expiration_date)
  VALUES ('TKN-PA-2026-000006', 'SN-PA-006', 'TitanQ-3',
          'IRIS', 6, 1, 2, 'ACTIVE',
          CURRENT_TIMESTAMP, CURRENT_DATE + INTERVAL '10 years')
  RETURNING token_id;
  -- assume returned token_id = 6

  INSERT INTO TokenLifecycleEvent (token_id, actor_agency_id, event_type, reason_code)
  VALUES (6, 1, 'ISSUED', 'INITIAL_ENROLLMENT'),
         (6, 1, 'ACTIVATED', 'POST_BIOMETRIC_ENROLLMENT');

  INSERT INTO TokenPermission (token_id, context_id, permission_level)
  VALUES (6, 1, 'VERIFY'),  -- BANKING
         (6, 4, 'VERIFY'),  -- TRAVEL
         (6, 6, 'VERIFY');  -- MOTOR_VEHICLE
COMMIT;
```

UC-4 (reserve activation after loss) requires two status changes on the same individual. The transitions are ordered: the lost token moves to its terminal status *first* (releasing it from the partial unique index's predicate), then the reserve transitions to `ACTIVE`. The partial unique index is never violated mid-transaction. This ordering eliminates the need for deferred-constraint semantics:

```
BEGIN;
  -- Demote the lost token
  UPDATE IdentityToken SET status = 'LOST' WHERE token_id = 3;
  INSERT INTO TokenLifecycleEvent (token_id, actor_agency_id, event_type, reason_code)
  VALUES (3, 1, 'LOST', 'HOLDER_REPORTED_LOSS');
  INSERT INTO RevocationList (token_id, revoked_by_agency_id, reason_code,
                              effective_date, published_location)
  VALUES (3, 1, 'LOST', CURRENT_DATE,
          'https://crl.idtoken.gov/2026/04');

  -- Promote a reserve token to ACTIVE on the same individual
  UPDATE IdentityToken
     SET status = 'ACTIVE',
         predecessor_token_id = 3,
         activation_sequence = 2,
         activated_date = CURRENT_TIMESTAMP
   WHERE individual_id = 2 AND status = 'RESERVE';
  INSERT INTO TokenLifecycleEvent (token_id, actor_agency_id, event_type, reason_code)
  SELECT token_id, 1, 'ACTIVATED', 'RESERVE_PROMOTION'
    FROM IdentityToken
   WHERE individual_id = 2 AND status = 'ACTIVE';
COMMIT;
```

Both transactions exercise the schema's distinguishing constraints (the partial uniqueness on `individual_id` where `status='ACTIVE'`, the FK chain across `IdentityToken`, `TokenLifecycleEvent`, and `RevocationList`, and the self-referential `predecessor_token_id` link) and demonstrate that the schema is operationally complete, not merely structurally correct.

7 SQL Implementation

The complete SQL script (`identity_token_system.sql`) implements the full schema, populates it with sample data, and defines a utility view. This section presents the schema overview, the most architecturally significant `CREATE TABLE` statements, and the sample-data summary. The full script is provided alongside this report and runs cleanly on PostgreSQL 14 or later.

7.1 Schema Overview

Table 14: SQL implementation summary.

Component	Count	Notes
Tables	12	10 entity tables + 2 junction tables
Foreign keys	18	Including 1 self-referential FK on `IdentityToken`
CHECK constraints	14	On every enumerated field across all tables
Composite primary keys	2	On `AgencyAlgorithmAuth` and `TokenPermission`
Partial unique constraints	1	Deferrable, on `(individual_id, status)`
Indexes	5	On high-query columns supporting verification and audit
Views	1	`ActiveTokens` for human-readable active-token summary
Sample rows	73	5 individuals, 6 agencies, 5 algorithms, 7 contexts, 5 tokens, 9 lifecycle events, 8 verification events, 5 device bindings, 2 blockchain anchors, 1 revocation, 9 authorization grants, 11 permission grants

7.2 Core Table: `IdentityToken`

This is the schema's central table. Its `CREATE TABLE` statement carries six foreign keys (one self-referential), the partial unique index that enforces one active token per individual, and `CHECK` constraints on three enumerated columns.

```
CREATE TABLE IdentityToken (
    token_id                    SERIAL PRIMARY KEY,
    token_value                 VARCHAR(128) NOT NULL UNIQUE,
    physical_serial             VARCHAR(64)  NOT NULL UNIQUE,
    hardware_model              VARCHAR(50),
    biometric_binding_type      VARCHAR(20)  NOT NULL
        CHECK (biometric_binding_type IN ('NONE','FINGERPRINT','FACE','IRIS')),
    biometric_enrolled_date     TIMESTAMP,
    enrollment_witness_agency_id INTEGER REFERENCES Agency(agency_id),
    liveness_check_type         VARCHAR(20)
        CHECK (liveness_check_type IN ('PASSIVE','ACTIVE_CHALLENGE','MULTI_MODAL')),
    individual_id               INTEGER NOT NULL REFERENCES Individual(individual_id),
    issuing_agency_id           INTEGER NOT NULL REFERENCES Agency(agency_id),
    algorithm_id                INTEGER NOT NULL REFERENCES
    ↪ CryptographicAlgorithm(algorithm_id),
    predecessor_token_id        INTEGER REFERENCES IdentityToken(token_id),
    activation_sequence         INTEGER DEFAULT 1,
    status                      VARCHAR(20) NOT NULL DEFAULT 'RESERVE'
        CHECK (status IN ('ACTIVE','RESERVE','DORMANT','REVOKED','LOST','EXPIRED')),
    issued_date                 TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    activated_date              TIMESTAMP,
    expiration_date             DATE
);

-- One-active-per-person invariant (partial unique index, see Appendix B):
CREATE UNIQUE INDEX uq_one_active_per_person
    ON IdentityToken (individual_id) WHERE status = 'ACTIVE';
```

7.3 Junction Table: TokenPermission

This is the resolution of the M:N PERMITTED relationship. The composite primary key on (token_id, context_id) both enforces the relational integrity of the junction and implements the application-level guarantee that a given (token, context) pair has exactly one permission level at any time.

```
CREATE TABLE TokenPermission (
    token_id          INTEGER NOT NULL REFERENCES IdentityToken(token_id),
    context_id        INTEGER NOT NULL REFERENCES VerificationContext(context_id),
    permission_level  VARCHAR(20) NOT NULL
        CHECK (permission_level IN ('READ','VERIFY','FULL')),
    granted_date      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (token_id, context_id)
);
```

7.4 Utility View: ActiveTokens

A single view supports the most common operational read pattern: human-readable summaries of every active token with its holder, issuer, and algorithm. The view enables UC-6 (algorithm migration audit) and the operational dashboard described in subsequent project work.

```
CREATE OR REPLACE VIEW ActiveTokens AS
SELECT  t.token_id,
        i.legal_name,
        a.name   AS issuing_agency,
        alg.name AS algorithm,
        t.status,
        t.activated_date
FROM    IdentityToken t
JOIN    Individual              i  ON t.individual_id     = i.individual_id
JOIN    Agency                  a  ON t.issuing_agency_id = a.agency_id
JOIN    CryptographicAlgorithm  alg ON t.algorithm_id     = alg.algorithm_id
WHERE   t.status = 'ACTIVE';
```

7.5 Sample Data Summary

The script populates the schema with realistic data representing a microcosm of the system in operation. Five individuals hold five tokens issued by three different federal and state agencies. The CryptographicAlgorithm catalog contains five entries (four post-quantum, one deprecated classical retained for migration tracking); the five sample tokens are all signed under post-quantum algorithms, while the classical algorithm appears in AgencyAlgorithmAuth only as a historical authorization record. Eight verification events span four contexts (banking, employment, travel, government benefits) at all three disclosure levels. Two tokens carry blockchain anchors (representing the optional DID architecture). One token has been superseded and appears in the revocation list. Eleven TokenPermission entries grant context-scoped verification rights; nine AgencyAlgorithmAuth entries record which agencies may issue, verify, or both under each algorithm.

8 Relational Algebra

The six queries below express representative use-case operations from the requirements analysis in formal relational algebra. Each query is annotated with the use case it serves and the SQL clause it would compile to.

Query 1: All Successful Banking Verifications for a Specific Individual

Serves UC-7 (warrant-authorized history review), restricted to the banking context.

$$\pi_{\text{event\_id, event\_timestamp, requestor\_location}}\Big(\sigma_{\text{outcome='SUCCESS'} \wedge \text{context\_type='BANKING'}}\big(\text{VerificationEvent} \bowtie \text{VerificationContext} \bowtie (\sigma_{\text{legal\_name='James Chen'}}\text{Individual} \bowtie \text{IdentityToken})\big)\Big)$$

Query 2: Active Tokens Using Post-Quantum Algorithms

Inverse of UC-6: returns the cohort that does *not* require migration. Useful for confirming progress of an in-flight migration.

$$\pi_{\text{token\_id, legal\_name, name}}\big(\sigma_{\text{quantum\_resistant=TRUE} \wedge \text{status='ACTIVE'}}\big(\text{IdentityToken} \bowtie \text{Individual} \bowtie \text{CryptographicAlgorithm}\big)\big)$$

Query 3: Agencies Authorized to Both Issue and Verify Using ML-DSA-65

Drives UC-1 step 1 (the issuing officer's authorization check) for the most-used post-quantum algorithm.

$$\pi_{\text{Agency.name, jurisdiction}}\big(\sigma_{\text{CA.name='ML-DSA-65'} \wedge \text{authorization\_type='BOTH'}}\big(\rho_{\text{CA}}(\text{CryptographicAlgorithm}) \bowtie \text{AgencyAlgorithmAuth} \bowtie \text{Agency}\big)\big)$$

Note on join condition: `CryptographicAlgorithm` and `Agency` both carry a `name` column. A pure natural join over the chain would match on `algorithm_id`/`agency_id` *and* on `name`, erroneously requiring the algorithm and agency names to be identical. The rename ρ_{CA} on `CryptographicAlgorithm` disambiguates: natural-join columns are `algorithm_id` (CA $\bowtie$ AAA) and `agency_id` (AAA $\bowtie$ Agency); the algorithm-name filter and the projected agency name are explicitly qualified.

Query 4: Device Bindings for All Active Tokens

Operational query for verifiers that accept digital presentation: enumerate which tokens currently project to which devices.

$$\pi_{\text{legal\_name, device\_type, device\_fingerprint, db.status}}\big(\rho_{\text{db}}(\text{DeviceBinding}) \bowtie_{\text{db.token\_id=it.token\_id}} \rho_{\text{it}}(\sigma_{\text{status='ACTIVE'}}\text{IdentityToken}) \bowtie \text{Individual}\big)$$

Note on join condition: both `DeviceBinding` and `IdentityToken` carry a `status` column with different semantics (binding lifecycle vs. token lifecycle). A pure natural join would incorrectly match on `status` as well as `token_id`. The explicit theta-join with rename (ρ) disambiguates: the join condition is `token_id` only, and the projected `status` comes from `DeviceBinding` (the binding's status, not the token's).

Query 5: Verification Volume by Context (Aggregate)

Drives capacity planning and detects unusual context skew that would suggest verifier abuse. Uses the aggregation operator γ (or $\mathcal{G}$ in some notations): ${}_{\text{group}}\gamma_{\text{aggregate}}$.

$$\pi_{\text{context\_type, vol}}\Big({}_{\text{context\_type}}\gamma_{\text{COUNT(event\_id)}\to\text{vol}}\big(\text{VerificationEvent} \bowtie \text{VerificationContext}\big)\Big)$$

Query 6: Token Succession Lineage (Self-Join on Predecessor)

Walks the `predecessor_token_id` self-reference one generation back. Returns each currently `ACTIVE` token paired with its immediate predecessor (if any). Repeated self-joins reconstruct longer lineages; relational algebra without recursion expresses the immediate parent only.

Let T_1, T_2 be two aliases for `IdentityToken`, with T_1 representing the current token and T_2 representing its predecessor.

$$\pi_{T_1\text{.token\_id},T_1\text{.activation\_sequence},T_2\text{.token\_id},T_2\text{.status}}\big(\sigma_{T_1\text{.status='ACTIVE'} \wedge T_1\text{.predecessor\_token\_id}=T_2\text{.token\_id}}\big(T_1 \times T_2\big)\big)$$

These six queries collectively exercise selection (σ), projection (π), natural join ($\bowtie$), Cartesian product ($\times$, Query 6), and grouped aggregation (γ, Query 5). They demonstrate the schema's capacity for high-volume verification analytics, cryptographic-migration prioritization, agency-authorization enforcement, device-binding enumeration, and audit-history reconstruction. These are the five operational dimensions on which the system will be evaluated.

9 Limitations and Open Problems

This design addresses several failure modes of the current siloed identity system, but a production deployment would face additional open problems that this database alone does not solve.

Catastrophic-loss risk. Reserve tokens and device bindings mitigate single-token loss, but a catastrophic event that destroys all of a holder's tokens and devices simultaneously (fire, theft, flood affecting both primary wallet and reserve storage) still leaves the holder without civic identity until reissuance. A production system would require a recovery protocol involving the issuing agency and out-of-band identity verification, with a defined grace period during which the holder retains access to essential services. **Issuer trust concentration.** Every token references an `issuing_agency_id`; if an issuing agency is compromised, deprecated, or politically disfavored, every token it issued becomes questionable. Historical precedent for identity-document-based denaturalization programs makes this a serious concern. A production system would require cryptographic diversity across issuers, a federation model with mutual recognition between independent authorities, and constitutional limits on issuer discretion. **Population coverage.** The schema assumes every individual holds exactly one active token; in practice newborns, undocumented residents, unhoused people, people without reliable access to reissuance infrastructure, and people whose biometrics do not register reliably with available hardware would all be outside the system. A production deployment would need either a tiered enrollment model or an accepted path for unregistered persons to participate in civic life without tokens.

Cryptographic migration during transitions. `CryptographicAlgorithm` includes a `deprecation_date` column, and the strategic rationale establishes that post-quantum signing is the schema's operating regime rather than a future migration target. The harder problem the schema does not yet model is how tokens transition between post-quantum primitives when one of them itself is later weakened or superseded: the second-generation migration problem, when ML-DSA or SLH-DSA is replaced by a successor algorithm, rather than the first-generation transition from classical primitives. A production system would require either simultaneous mass reissuance, or a multi-signature scheme where tokens can accept signatures from multiple algorithms during transition periods, or both. The substrate-layer argument (Appendix E) is the deeper context within which this problem must be solved. **Compulsion resistance.** Biometric binding prevents casual token theft but does not prevent compelled use; a border officer, law enforcement agent, or hostile actor can physically compel fingerprint or iris presentation. On-device biometrics address unauthorized use but not coerced use, and a production system would need additional protections (duress codes, attestation delays, remote revocation) at the hardware level, outside the scope of this relational schema. Quantum-observer binding (Appendix F) is the direction this points toward. **Centralized trust assumption.** The current schema assumes a federated but fundamentally centralized trust model: issuing agencies are authoritative, and token validity is determined by reference to agency and algorithm records. An alternative direction that future work could explore is decentralized-identifier (DID) anchoring, in which each token is bound to a cryptographic commitment recorded on a post-quantum-capable distributed ledger rather than solely to an issuing-agency record; the relational schema would remain useful as the off-chain event and audit layer.

These problems are acknowledged as open. The schema is structurally complete and provides a coherent foundation for the web interface and testing work that builds on it. A production deployment would require substantive additions beyond the database layer.

Appendix A Token Lifecycle State Machine

The `status` column on `IdentityToken` admits six values: `RESERVE`, `ACTIVE`, `DORMANT`, `REVOKED`, `LOST`, and `EXPIRED`. The state machine in Figure 3 formalizes the legal transitions, with `ACTIVE` at the center and the five other states arranged at the vertices of a regular pentagon. `ACTIVE` is the operational hub: every legal transition originates at `ACTIVE` or terminates at `ACTIVE` through `RESERVE`.

Constraint Implications

The state machine encodes invariants that the schema's `CHECK` constraints alone cannot express. No transition from `REVOKED`, `LOST`, or `EXPIRED` returns to `ACTIVE`; a new token must be issued. No direct transition from `RESERVE` to a terminal state without first passing through `ACTIVE`: an unactivated reserve is destroyed by reissuance, not revoked. Exactly one `ACTIVE` token per `individual_id` at any moment, enforced by the partial unique index. The transition `ACTIVE` → `DORMANT` occurs in coordination with another token's transition `RESERVE` → `ACTIVE`; the application-layer ordering of these two updates (terminal status first, then promotion) keeps the partial unique index satisfied throughout the transaction. A production deployment would enforce these rules in a database trigger or in the application layer.

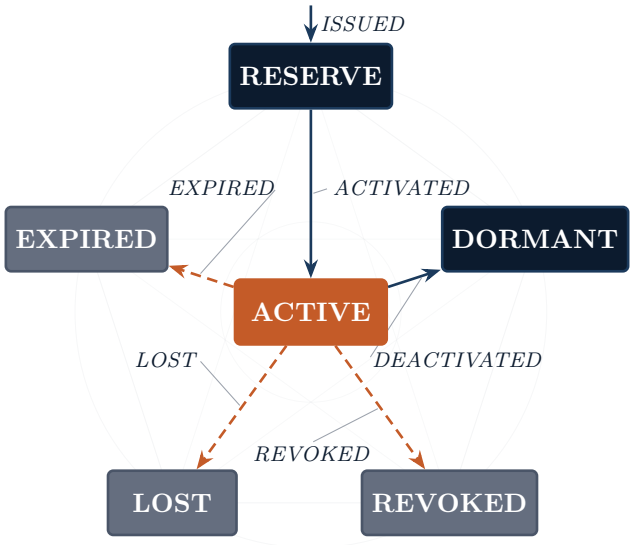

Figure 3: Token lifecycle state machine. `ACTIVE` at the center is the only operational state; the five surrounding states are reached by transitions from `ACTIVE` except `RESERVE`, which is the sole entry point from issuance. Solid arrows are non-terminal transitions; dashed arrows in accent are terminal (the token never returns from `REVOKED`, `LOST`, or `EXPIRED`). `DORMANT` is non-terminal in principle (audit access is preserved) but tokens never re-enter `ACTIVE` from `DORMANT`; succession runs through a fresh `RESERVE`-to-`ACTIVE` chain on the successor token, with `predecessor_token_id` carrying the lineage.

Production Enforcement of the State Machine

The schema expresses three of the state machine's invariants directly: the domain of `status` is constrained by a `CHECK` clause that admits only the six legal values; the at-most-one-`ACTIVE`-per-individual invariant is enforced by the partial unique index on `individual_id` where `status='ACTIVE'`; the requirement that every state change be audited is enforced operationally by

pairing each `UPDATE IdentityToken` with a corresponding `INSERT INTO TokenLifecycleEvent` in the same transaction. These three are sufficient to prevent the most damaging illegal states.

The *transitions* themselves are not directly expressible as schema-level constraints because SQL has no first-class concept of state-machine arrows. A production deployment would enforce them through one of three mechanisms. **Database triggers** on `IdentityToken` can reject `UPDATE` statements whose `(OLD.status, NEW.status)` pair is not in the legal-transitions set; this is the strongest option because it guarantees enforcement regardless of which client connects to the database. **Stored procedures** that wrap each transition (`activate_reserve`, `revoke_token`, `record_loss`) can be granted to application roles while direct `UPDATE` privileges on `IdentityToken` are revoked; this is the most auditable option because every transition becomes a named, logged operation. **Application-layer enforcement** in the verifying service can mirror the state machine in code; this is the least defensible option in isolation, since a misconfigured second client can violate the invariants, but it is acceptable when paired with one of the database-level mechanisms. The recommendation is triggers for the hard edges (no return from terminal states) and stored procedures for the routine transitions, with the application layer treating both as opaque.

Appendix B Indexing Strategy and Query-Plan Notes

Table 15: Secondary indexes and the queries they support.

Index	Cardinality	Motivating Query / Access Pattern
`idx_identitytoken_individual`	High (1 per holder)	UC-7 history reconstruction; the partial unique constraint also benefits from this index for fast active-token lookup.
`idx_identitytoken_status`	Low (6 distinct values)	The `ActiveTokens` view scans tokens with `status='ACTIVE'` only; index used for filtering before join.
`idx_verificationevent_timestamp`	Very high	Capacity-planning queries (Q5), audit history retrieval (UC-7), and time-bounded subscription windows.
`idx_verificationevent_context`	Low (7 distinct values)	Per-context analytics (Q5); per-context fraud detection.
`idx_tokenlifecycle_token`	Moderate	Reconstructing a single token's lifecycle on demand (UC-4 reserve activation, UC-7 audit).

Notes on Query Plans

Three of the six relational-algebra queries deserve specific attention:

Query 1 (UC-7 banking history). Filters `VerificationEvent` on `outcome='SUCCESS'`, joins `VerificationContext` on `context_id`, and joins `Individual` through `IdentityToken` on `individual_id`. The planner should first restrict by `outcome` (column not currently indexed; **candidate for an additional index** if banking-history queries become frequent), then use `idx_verificationevent_context` for the context join, then nested-loop-join through the token to the individual. Estimated cost: $O(N_v \cdot \log N_t)$ where N_v is verification-event count and N_t is token count.

Query 5 (verification volume by context). Pure aggregation over `VerificationEvent` grouped by `context_id`. The planner uses `idx_verificationevent_context` to drive a hash-aggregate. With 7 distinct context values, the aggregate is bounded; the join to `VerificationContext` for human-readable labels is a 7-row lookup. Cost: $O(N_v)$.

Query 6 (succession lineage). Self-join on `IdentityToken` via the nullable `predecessor_token_id`. The planner uses the primary key on `token_id` for the inner

side. A future enhancement is a dedicated `idx_identitytoken_predecessor` index; with the current sample-data size it is unnecessary, but a production deployment with hundreds of millions of tokens should add it.

Partitioning Recommendation (Future Work)

`VerificationEvent` is the only table that grows without bound in normal operation. A national deployment would generate on the order of 10^8 to 10^9 rows per year. Two partitioning strategies are candidates:

- **Range partitioning on `event_timestamp`**, monthly. Old partitions can be archived to cold storage and detached from the live table without touching the rest of the schema. This is the recommended primary strategy.
- **List partitioning on `context_id`**, secondary. Verification volume is unlikely to be uniform across contexts (banking dominates), so context-list partitioning helps with parallel-scan performance for context-specific queries.

PostgreSQL 14+ supports both. A composite partitioning scheme (range on timestamp, then list on context within each monthly partition) is feasible but not recommended initially.

Appendix C Existing National Identity Systems

The Identity Token System is not the first attempt at a unified digital credential. Five existing systems represent the design space the present schema must situate itself within. Each is summarized below, followed by a comparative table.

A federated national eID with a data-exchange backbone. The most mature national digital-identity infrastructure in production issues every resident a state-issued chip card with two PKI key pairs (authentication and signing) under a state-operated certificate authority. Behind the card sits a federated data-exchange backbone that lets agencies verify identity and exchange data without centralizing it. The architecture has operated for over two decades and is the closest live analogue to the present design, but its PKI is not post-quantum.

ISO 18013-5 Mobile Driver's License (mDL). An international standard for mobile-phone-based driver's licenses, finalized in 2021 and adopted by several US states (Arizona, Iowa, Maryland, Mississippi, Oklahoma, Utah, Colorado, others) plus Australia and Argentina. The mDL standard supports selective disclosure but stores the credential on a phone, not a hardware token, and does not address replacement contexts beyond driving authority. Quantum resistance is a roadmap item.

India Aadhaar. The world's largest biometric identification system, with over 1.3 billion enrollments. Aadhaar centralizes biometric templates (fingerprint and iris) in a national database (the Central Identities Data Repository), which is the precise architectural choice the present design rejects. Verification is done by sending biometric samples to the centralized database, which returns a yes/no response. The system has been the subject of substantial constitutional litigation in India; the 2018 Puttaswamy decision restricted but did not abolish it.

EU eIDAS 2.0 / European Digital Identity Wallet. The EU's 2024 regulation establishing a federated digital-identity framework across all 27 member states, with the European Digital Identity Wallet (EUDI Wallet) as the user-facing artifact. Architecturally federated rather than centralized: each member state issues credentials, and the wallet aggregates them. Selective disclosure is mandated. Post-quantum migration is in roadmap discussions but not yet specified.

W3C Verifiable Credentials and Decentralized Identifiers. A standards stack rather than a deployment: `did:` URI scheme, JSON-LD credential format, BBS+ signatures for selective disclosure. Adopted by Microsoft Entra Verified ID, IBM Digital Credentials, and

several blockchain projects. The standards are issuer-agnostic; the present design's optional `BlockchainAnchor` entity is the bridge to this ecosystem.

Table 16: Comparison of identity-system architectures.

System	Trust Model	Biometric	Revocation	Post-Quantum
Federated national eID	Federated state CA	Optional, on-device	OCSP / CRL	Classical (RSA, ECDSA)
ISO 18013-5 mDL	Issuer-signed	On-device	Status list	Classical; PQ on roadmap
India Aadhaar	Centralized (UIDAI)	Required; central template	Lock flag	Classical
EU eIDAS 2.0 EUDI	Federated, per-MS issuers	Optional, on-device	Status list	Classical; PQ planned
W3C VC / DID stack	Decentralized	N/A (out of scope)	Revocation registry	PQ-capable (BBS+)
Identity Token (this work)	**Federated + DID anchor**	**On-device only**	`RevocationList` + `status`	**Native PQ (ML-DSA, SLH-DSA)**

Where the Present Schema Sits

The Identity Token System borrows the federated trust model from a mature federated national eID and eIDAS, the on-device biometric model from mDL, the optional decentralized-identifier anchoring from W3C VC, and the post-quantum signature primitives from NIST. It rejects Aadhaar's centralized-biometric model and the classical-cryptography posture of every other system (NIST having formally finalized post-quantum standards in August 2024). The deliberate distinguishing feature is the `disclosure_level` column on `VerificationEvent`: none of the five comparison systems carries a comparable schema-level mechanism for preventing the verification log from functioning as a surveillance database. mDL and EUDI permit selective disclosure at the protocol level but record verifications at full granularity in the verifier's logs; the present schema makes the disclosure level a property of the persisted record itself.

Patterns Across the Field

Reading the five comparison systems together reveals two convergences and one open question. **Federation has won.** Aadhaar is the only large-scale centralized system in the comparison; every other architecture (the federated national eID, mDL, eIDAS 2.0, W3C VC) federates issuance across multiple authorities. The post-Snowden, post-Equifax design environment treats centralized biometric repositories as inherently untenable, regardless of operator intent. **On-device biometric handling has won.** mDL, eIDAS 2.0, and the present design all keep biometric templates in the holder's device and reduce verification to a yes/no signal sent to the verifier. Aadhaar's centralized template repository is the architectural outlier even within India's own roadmap discussions. The open question is **post-quantum migration**. That national eID is on classical cryptography with no announced migration plan; mDL has post-quantum on its roadmap but no schedule; eIDAS 2.0 has it in active discussion but no specified target date; W3C VC supports BBS+ signatures, which are PQ-capable but not PQ-mandatory. Polaris's choice to make post-quantum primitives the only signing option for new tokens is the outlier in 2026; by 2030 it should be the median.

Appendix D Threat Model (STRIDE)

STRIDE is Microsoft's threat-classification framework: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege. It is the right framework for

this analysis because each of its six categories maps directly to a class of database-layer attack, and the framework is exhaustive over the threats that the schema itself can defend against. Threat models scoped at the application or network layer (PASTA, OCTAVE, attack trees) would miss the schema-level mitigations this design relies on; STRIDE is structured to surface exactly them.

The analysis below scopes the threat model to the database layer. This is a deliberate restriction: the schema can guarantee what it is queryable for and what it refuses to store, but it cannot guarantee what happens before data arrives at the database (compelled biometric presentation, issuer-key compromise) or what happens after data leaves it (exfiltration through a privileged client). The three categories of threat outside the schema's reach are enumerated in the subsection that follows the table. The *Schema-Level Mitigation (and Residual Risk)* column is therefore the operative one: it states what the schema does and, where present, what it does not do even when it does its job correctly.

Table 17: STRIDE analysis of the Identity Token System schema.

Category	Threat	Schema-Level Mitigation (and Residual Risk)
Spoofing	Adversary presents a stolen or counterfeit token to a verifier.	On-device biometric binding gates token use; `biometric_binding_type` on `IdentityToken` records the method. Template never enters the database. Residual risk: compelled biometric presentation.
Tampering	Adversary modifies a token's signature, status, or permissions.	Tokens are signed under a NIST PQ algorithm (`ML-DSA`, `SLH-DSA`) recorded in `CryptographicAlgorithm`; database-level changes require write access addressed by deployment hardening.
Repudiation	A verifier or issuer denies an action they performed.	`TokenLifecycleEvent` is an append-only audit trail with `actor_agency_id`; `VerificationEvent` records `requesting_agency_id`. The append-only invariant is by convention and tooling.
Information Disclosure	Verification log becomes a surveillance database, or biometric templates are exfiltrated.	`disclosure_level` on `VerificationEvent` controls persistence: `ZERO_KNOWLEDGE` stores only a commitment; `SELECTIVE` discloses specified attributes; `FULL` is restricted to warrant-authorized verification. Templates never stored centrally.
Denial of Service	Adversary overwhelms the verification layer or relies on stale revocation data.	Indexes (Appendix B) keep single-row lookups at $O(\log N)$; rate limiting is deferred to deployment. `RevocationList` carries `published_location` for verifier freshness checks.
Elevation of Privilege	An agency issues or verifies under an algorithm it is not authorized to use.	`AgencyAlgorithmAuth` enforces `ISSUE` / `VERIFY` / `BOTH` per (agency, algorithm) pair; `Agency` carries `authorization_level` (1–5) for coarse-grained policy.

Threats Outside the Schema's Reach

Three categories of threat the database layer cannot mitigate. *Compelled biometric presentation:* an actor with physical access can compel fingerprint, iris, or face presentation; the schema cannot distinguish coerced from voluntary use, and hardware-level features (duress codes, attestation delays) are needed. *Issuer compromise:* if an issuing agency's signing keys are compromised, every token it issued is suspect; mitigation requires cryptographic diversity across issuers and out-of-band attestation channels. *Cryptanalytic advance:* should ML-DSA or SLH-DSA be broken before scheduled migration, the entire token population is at risk; `deprecation_date` enables proactive planning, but multi-signature transitional states are not yet modeled (noted in Limitations).

Appendix E Why Identity Cannot Outrun Its Primitives

Section 1.1 justified treating post-quantum signing as the schema's operating regime through Mosca's inequality and the time horizon of identity data. This appendix formalizes the deeper architectural argument: cryptographic primitives constitute the *substrate* of digital sovereignty, and every higher-level property of an identity system is derivative of the primitives it sits on top of. Identity infrastructure is conventionally drawn as a layered stack: hardware roots of trust, cryptographic primitives, authenticated communications, identity and authentication, data sovereignty, and at the top, algorithmic governance. The schema sits at the identity and authentication layer. Every higher-level property it implements (authenticity of signed credentials, integrity of audit trails, binding of biometric attestations, validity of zero-knowledge proofs) is derivative of properties at the cryptographic substrate two layers below. Change the primitive and the property changes; compromise the primitive and the property has no referent. The visible operations of an identity system are not buildings on a foundation but emanations of the substrate, downstream of a one-way causal descent through every authenticated, signed, or attested operation in the stack.

Classical public-key cryptography rests on factorization and discrete-log hardness. Shor's algorithm (1994) demonstrated that a sufficiently large quantum computer solves both in polynomial time: the hardness assumption is not weakened, it is annihilated. Two consequences follow from this single mathematical fact, and they apply to identity infrastructure asymmetrically. **Signatures**: every credential signed today under ECDSA or RSA can be *forged* once Shor-capable hardware reconstructs the issuing key, which means the binding between a biometric anchor and a holder's legal identity becomes asserted by whichever adversary recovers the key, not only by the original issuer. **Encrypted material**: any ciphertext intercepted today and stored under classical key exchange (TLS handshakes, sealed identity proofs, archived authentication transcripts) sits in a latent pre-collapse state, decryptable when the substrate transitions; harvest-now-decrypt-later is the operational name for this pattern. The transition is not an attack but a phase transition: the cryptographic past does not become vulnerable when a cryptographically relevant quantum computer enters service, it becomes *determined*. The schema's signed bindings and any associated wire-protocol transcripts are both implicated.

Three implications follow. **First**, the schema cannot be built defensively against a post-rendering substrate: the transition is discontinuous, and no schema-level mechanism recovers what is lost when a primitive becomes meaningless. The correct response is to architect for the post-rendering substrate from the start, which is what this design does by refusing to issue new tokens under classical algorithms. **Second**, sovereignty at the database layer is derivative of sovereignty at the primitive layer; a schema implementing the strongest privacy controls over signatures from an adversary-authored algorithm produces compliance, not sovereignty. The commitment to NIST-finalized primitives (FIPS 203, 204, 205) is a sovereignty stance, not a technical preference. **Third**, the database layer is not the only layer at which an identity system can fail; the human substrate (credentialed operators) and the operational substrate sit outside the relational schema's scope. The schema can refuse to amplify their failure modes but cannot substitute for human-level and policy-level controls.

Appendix F Beyond Biometric Binding

The current schema records biometric binding through the `biometric_binding_type` column on `IdentityToken`, which admits four values: `NONE`, `FINGERPRINT`, `FACE`, and `IRIS`. This is the strongest binding mechanism that production hardware can deliver in 2026. It is not the strongest binding mechanism that is possible in principle. This appendix sketches two extensions of the binding model that the schema would need to accommodate as the underlying hardware advances:

genomic binding as a near-term hardware extension, and **quantum-observer binding** as a far-future architecture that closes the binding chain at the only link no adversary can copy.

F.1 Genomic Binding (Near-Term Hardware Extension)

A genomic binding extension would add `GENOMIC` to the `biometric_binding_type` domain and introduce a `GenomicCommitment` entity holding zero-knowledge proofs of uniqueness over the holder's genome. The raw genome would never enter the database; only homomorphic commitments and ZK proofs would. The owner's secure enclave would hold the sequence locally and produce per-verification proofs. A human genome carries roughly 3×10^9 base pairs, of which $\sim 4\text{–}5 \times 10^6$ vary between individuals — more than sufficient entropy to defeat any forgery attack at scale.

The architectural problem is that **genomic data can still be stolen**. The threat surface is well-documented: consumer-genomics breaches (23andMe, October 2023, ∼7M profiles), surreptitious public-space collection, state-level exfiltration from medical or research databases, and synthesis attacks reconstructing the profile from partial-relative data. Once the profile is in adversarial hands, the binding is broken: the owner's body is no longer the only entity that can produce a valid proof. This is the fundamental ceiling of any binding tied to *biological data* — the data is copyable.

F.2 Quantum-Observer Binding (Speculative)

The next horizon abandons binding to biological *data* and binds instead to the *act* of observation. Measurement in quantum mechanics is not passive; the resulting collapse is irreproducible by anyone who did not perform it. The cryptographic realization uses quantum entanglement: the token's authenticating key material is prepared as a Bell-type entangled state, half held in the token's secure quantum register and half projected through a brain-computer interface tuned to the owner's neural quantum signature. The state cannot be duplicated (no-cloning theorem) and cannot be shared (monogamy of entanglement). When the owner directs attention through the BCI, the measurement collapses the entangled pair into the correct decryption key for that authentication moment; any other observer's measurement collapses it into noise. Under the contested but seriously-discussed Orch-OR hypothesis (Penrose-Hameroff), quantum-coherent processes in microtubular cytoskeletons make the collapse pattern specific to a brain's connectome — a fingerprint no apparatus simulates without instantiating an equivalent neural architecture.

None of this is engineered in 2026; the architecture is a research direction. Returning to the threat model in Appendix D: of the three residual risks, quantum-observer binding directly addresses compelled biometric presentation. Under stress the neural signature shifts and the state decoheres without yielding the key; the binding is not merely *tied to* the holder, it *is* the holder. The three generations (biometric, genomic, quantum-observer) form a nested progression that the present schema would resolve through a hypothetical `TokenBinding` junction table replacing the current scalar `biometric_binding_type` column; everything else is hardware, physics, and time.